МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Алгоритмы и структуры данных»

Тема: Реализация и исследование алгоритма сортировки TimSort

Студент гр. 5387 _____ Степанов Л. А.

Преподаватель _____ Суворов А. А.

Санкт-Петербург
2026

Оглавление

Цель работы 3

Основные теоретические положения 5

Выполнение работы 6

Pipeline данных 11

Исследование 13

Вывод 22

ПРИЛОЖЕНИЕ А 23

Исходный код программы 23

ПРИЛОЖЕНИЕ Б 36

Результаты тестирования 36

Цель работы

Реализовать гибридный алгоритм сортировки TimSort для массива целых чисел по убыванию модуля и исследовать, как время его работы зависит от размера данных и от параметра min_run.

Задание

Имеется массив данных для сортировки int arr[] размера n. Необходимо отсортировать его алгоритмом сортировки TimSort по убыванию модуля.

Так как TimSort - это гибридный алгоритм, содержащий в себе сортировку слиянием и сортировку вставками, то нужно использовать оба этих алгоритма. Поэтому нужно выводить разделённые блоки, которые уже отсортированы сортировкой вставками.

Кратко алгоритм сортировки можно описать так:

- вычисление min_run по размеру массива n (для упрощения отладки n уменьшается, пока не станет меньше 16, а не 64);
- разбиение массива на частично-упорядоченные (в т.ч. и по убыванию) блоки длины не меньше min_run;
- сортировка вставками каждого блока;
- слияние каждого блока с сохранением инварианта и использованием галопа (галоп начинать после 3-х вставок подряд).

После успешного решения задачи нужно провести исследование сортировки на различных размерах данных (10/1000/100000), сравнив полученные результаты с теоретической оценкой (для лучшего, среднего и худшего случаев), и разного размера min_run. Для исследования используется стандартный алгоритм вычисления min_run, а галоп начинается после 7-ми вставок подряд.

Формат ввода:

Первая строка содержит натуральное число n - размерность массива, следующая строка содержит элементы массива через пробел.

Формат вывода:

Выводятся разделённые блоки для сортировки в формате «Part i: *отсортированный разделённый массив*». Затем для каждого слияния выводится количество вхождений в режим галопа и получившийся массив в формате «Gallops i: *число вхождений в галоп*» и «Merge i: *итоговый массив после слияния*». Последняя строчка содержит финальный результат сортировки массива с надписью «Answer: ».

Пример (min_run = 10):

Ввод:

20

1 -2 3 -4 5 6 -7 -8 9 -10 11 -10 -9 8 7 -7 -6 6 5 4

Вывод:

Part 0: 11 -10 9 -8 -7 6 5 -4 3 -2 1

Part 1: -10 -9 8 7 -7 -6 6 5 4

Gallops 0: 0

Merge 0: 11 -10 -10 9 -9 -8 8 -7 7 -7 6 -6 6 5 5 -4 4 3 -2 1

Answer: 11 -10 -10 9 -9 -8 8 -7 7 -7 6 -6 6 5 5 -4 4 3 -2 1

Основные теоретические положения

TimSort - гибридный алгоритм сортировки, который в 2002 году придумал Тим Питерс для языка Python. Алгоритм опирается на простое наблюдение: реальные данные почти никогда не бывают полностью случайными - в них есть уже упорядоченные куски. Такие куски называют ранами (run), и TimSort старается их найти и использовать, а не сортировать всё заново.

Работа алгоритма делится на два этапа. Сначала массив разбивается на раны длины не меньше min_run: если естественный упорядоченный кусок короче, к нему добавляются следующие элементы, и весь блок досортировывается вставками. На маленьких массивах сортировка вставками работает быстро - у неё почти нет накладных расходов, она устойчива и на уже упорядоченных данных делает всего n - 1 сравнение. Ран, идущий в обратном порядке, просто разворачивается за O(k). Важно, чтобы при этом он был строго обратным - иначе разворот переставил бы равные элементы и сломал устойчивость.

Число min_run подбирается так, чтобы количество блоков n / min_run было степенью двойки или чуть меньше её. Для этого n сдвигается вправо, пока не станет меньше порога (в стандартном варианте 64), а если хотя бы один выдвинутый бит был единицей, к результату прибавляется 1. Тогда на этапе слияния куски сливаются примерно поровну, как в идеальной сортировке слиянием.

На втором этапе раны кладутся в стек и сливаются. Чтобы сливались куски соизмеримого размера, для трёх верхних ранов X, Y, Z (Z - верхний) поддерживается инвариант $|X| > |Y| + |Z|$ и $|Y| > |Z|$. Из него следует, что длины ранов в стеке растут не медленнее чисел Фибоначчи, поэтому глубина стека - O(log n), а каждый элемент участвует в O(log n) слияниях.

Выполнение работы

В основе решения лежит вектор std::vector<int> и набор отдельных функций, каждая из которых отвечает за свой шаг алгоритма: вычисление min_run, поиск рана, сортировку вставками, галоп, слияние и поддержку инварианта стека. Сравнение элементов при сортировке вставками и слиянии идёт через функцию before, поэтому порядок сортировки (по убыванию модуля) задаётся в одном месте.

Описание программы

Программа читает n и массив, вызывает tim_sort и по ходу работы печатает разделённые блоки, результаты слияний и ответ. Весь алгоритм находится в файле src/main.cpp - это ровно тот код, который отправлялся в тестирующую систему. Все вспомогательные файлы вынесены в папку src/modules: тесты (tests.cpp), нагрузочный тест (benchmark.cpp) вместе с копией алгоритма для замеров (timsort.h), таблица результатов замеров (results.csv) и скрипт построения графиков (plot_benchmark.py).

Глобальные параметры:

- const int MINRUN_LIMIT = 16 - порог, до которого уменьшается n при вычислении min_run.
- const int GALLOP = 3 - после скольких побед одной стороны подряд начинается галоп.
- int mergeCounter - номер текущего слияния для строк «Gallops i» и «Merge i».
- struct Run { int len; int start; } - описание рана: длина и позиция начала в массиве.

Для исследования используется копия алгоритма из файла src/modules/timsort.h. Логика в ней та же, но MINRUN_LIMIT и GALLOP сделаны изменяемыми, чтобы мерить и вариант из условия, и стандартный (порог 64, галоп после 7 побед). Кроме того, в копию добавлены параметр FIXED_MINRUN (если он больше нуля, min_run берётся равным ему, а не считается по формуле), флаг PRINT (в нагрузочном тесте вывод

отключается) и счётчики comparisons и gallopsTotal - число сравнений и входов в галоп. Совпадение копии с main.cpp проверено: на 500 случайных массивах вывод обеих программ одинаков.

Функции:

- bool before(int a, int b) - true, если a должен стоять строго раньше b, то есть |a| > |b|.
- int calc_minrun(int n) - вычисляет min_run по размеру массива.
- void reverseRange(vector<int>& arr, int lo, int hi) - разворачивает отрезок arr[lo..hi) двумя указателями.
- int find_run(vector<int>& arr, int lo, int n) - находит естественный ран, начинающийся в lo, и возвращает позицию сразу за ним.
- void insertion_sort(vector<int>& arr, int lo, int hi) - устойчивая сортировка вставками отрезка arr[lo..hi).
- bool goesFirst(int x, int key) - вспомогательная для галопа: true, если |x| ≥ |key|.
- int gallop(const vector<int>& arr, int from, int end, int key) - считает, сколько элементов подряд из arr[from..end) идут раньше key.
- int merge_runs(vector<int>& arr, int lo, int mid, int hi) - сливает соседние раны arr[lo..mid) и arr[mid..hi) и возвращает число входов в галоп.
- void mergeAt(vector<int>& arr, vector<Run>& stack, int k) - сливает раны stack[k] и stack[k + 1], печатает «Gallops» и «Merge» и склеивает две записи стека в одну.
- void merge_collapse(vector<int>& arr, vector<Run>& stack) - восстанавливает инвариант стека после добавления нового рана.
- void mergeForceCollapse(vector<int>& arr, vector<Run>& stack) - в конце сливает всё, что осталось в стеке.
- void tim_sort(vector<int>& arr, int n) - сама сортировка: делит массив на блоки, печатает их, а затем сливает.

Детальное описание функций:

int calc_minrun(int n)

Число n сдвигается вправо, пока не станет меньше MINRUN_LIMIT; каждый выдвинутый бит попадает в r через r |= n & 1. В конце возвращается n + r. Например, при пороге 16: для n = 20 получаем 20 → 10 и min_run = 10, а для n = 33 - 33 → 16 → 8, выдвинулась единица, и min_run = 8 + 1 = 9.

int find_run(vector<int>& arr, int lo, int n)

Если с позиции lo остался один элемент, это ран длины 1. Иначе сравниваются первые два элемента. Если модуль строго вырос (|arr[lo]| < |arr[lo + 1]|), ран идёт в обратном для нас порядке: указатель hi двигается, пока рост строгий, а затем отрезок разворачивается через reverseRange. Если модуль не вырос, ран уже в нужном порядке, и hi двигается, пока модули не возрастают (равные допускаются). Возвращается hi - позиция сразу за раном, с неё начинается следующий.

void insertion_sort(vector<int>& arr, int lo, int hi)

Обычная сортировка вставками: слева от i всё уже отсортировано, ключ key = arr[i] вынимается, и все элементы, перед которыми он должен стоять строго раньше, сдвигаются на одну позицию вправо. Сдвиг идёт только при before(key, arr[j]), то есть при строго большем модуле ключа, поэтому равные элементы не обгоняют друг друга, и сортировка устойчива. Первые элементы блока - это естественный ран, и для них цикл не делает ни одного сдвига.

int gallop(const vector<int>& arr, int from, int end, int key)

Если первый элемент не идёт раньше ключа, сразу возвращается 0. Иначе выполняется экспоненциальный поиск: проверяются смещения 1, 3, 7, 15, … (bound = bound * 2 + 1), пока элемент ещё подходит; last_ok запоминает последнее подходящее смещение. Затем в интервале (last_ok, bound] бинарным поиском ищется первое неподходящее смещение - оно и равно числу элементов, которые можно забрать одним блоком.

int merge_runs(vector<int>& arr, int lo, int mid, int hi)

Оба рана копируются во временные векторы left и right, а результат пишется обратно в arr начиная с lo. На каждом шаге сравниваются left[i] и right[j]: если правый элемент не идёт строго раньше левого, берётся левый - при равных модулях это сохраняет порядок. Параллельно ведутся счётчики побед подряд count_left и count_right; победа одной стороны обнуляет счётчик другой. Когда один из счётчиков доходит до GALLOP, увеличивается число галопов, и, если в обоих массивах ещё остались элементы, gallop находит длину блока, который сразу копируется. После галопа оба счётчика обнуляются. Когда один из массивов заканчивается, хвост второго докопируется без сравнений.

void merge_collapse(vector<int>& arr, vector<Run>& stack)

Пока в стеке больше одного рана, проверяется инвариант. Если нарушено $|X| \le |Y| + |Z|$ или (для четырёх ранов) $|W| \le |X| + |Y|$, сливается Y с меньшим из соседей: X и Y, если $|X| < |Z|$, иначе Y и Z. Если нарушено только $|Y| \le |Z|$, сливаются Y и Z. Когда все условия выполнены, цикл заканчивается.

void tim_sort(vector<int>& arr, int n)

Сначала вычисляется min_run. Затем массив проходится слева направо: find_run находит естественный ран, если он короче min_run, блок дополняется до min(lo + min_run, n), сортируется вставками и печатается как «Part i», а пара (длина, начало) запоминается в списке runs. После этого раны по одному кладутся в стек, и после каждого вызывается merge_collapse. В конце mergeForceCollapse сливает остатки, и печатается ответ.

Особенности, выявленные при проверке

При сдаче в тестирующую систему выяснились три детали, которые не следуют из условия напрямую, но нужны, чтобы вывод совпал с эталоном:

- сначала печатаются все блоки «Part», а уже потом все слияния - поэтому деление на блоки и слияние сделаны двумя отдельными проходами;
- вход в галоп засчитывается всегда, когда серия доходит до GALLOP, даже если одна из половин к этому моменту закончилась;
- галоп по правому рану забирает элементы с модулем, равным ключу ($|x| \geq |key|$). Из-за этого в примере 2 из условия элемент 6 из правого рана оказывается раньше -6 из левого, то есть галоп по правой стороне нарушает устойчивость. Это поведение эталона: со строгим сравнением пример не совпадает. Все остальные шаги алгоритма устойчивы: при вставках и обычном слиянии элементы с равными модулями не обгоняют друг друга.

Чтобы проверить, что всё работает правильно, написан набор тестов на Google Test (src/modules/tests.cpp). Тесты подключают сам файл main.cpp, а не копию алгоритма: на время подключения функция main переименовывается, поэтому проверяется ровно тот код, который отправлялся в систему. Вывод программы совпадает с ожидаемым на всех тестах системы.

Pipeline данных

Проследим по шагам, как сортируется массив из примера 2 условия (порог 16, галоп после 3 побед подряд). Здесь хорошо видны все этапы: разворот рана, стек, слияние и вход в галоп.

Начальное состояние

n = 16

arr: [-1, 2, 3, 4, 5, -6, 7, 8, -8, -8, 7, -7, 7, 6, -5, 4]

1. calc_minrun(16);

16 ≥ 16, поэтому n сдвигается вправо: 16 → 8, выдвинутый бит равен 0, r = 0. 8 < 16 - цикл заканчивается, min_run = 8 + 0 = 8.

2. find_run(arr, 0, 16);

Первые два элемента: |2| > |-1|, модуль строго вырос - ран строго возрастающий. Указатель hi идёт, пока рост строгий: |-1| < |2| < |3| < |4| < |5| < |-6| < |7| < |8|, а |-8| уже не больше |8|, поэтому hi = 8. Отрезок arr[0..8) разворачивается.

Состояние: arr[0..8): [8, 7, -6, 5, 4, 3, 2, -1]

Длина рана 8 = min_run, дополнять не нужно. insertion_sort проходит по блоку, но ничего не сдвигает - блок уже упорядочен. Печатается «Part 0: 8 7 -6 5 4 3 2 -1».

3. find_run(arr, 8, 16);

Первые два элемента: |-8| = |-8|, модуль не вырос - ран невозрастающий. Указатель идёт, пока модули не растут: 8, 8, 7, 7, 7, 6, 5, 4 - и доходит до конца массива, hi = 16. Ран уже в нужном порядке, разворот не нужен. Печатается «Part 1: -8 -8 7 -7 7 6 -5 4».

Состояние: runs: (len 8, start 0), (len 8, start 8)

4. Стек ранов;

Первый ран кладётся в стек: [8]. В стеке одна запись, сливать нечего. Кладётся второй: [8, 8]. Для Y = 8 и Z = 8 нарушено условие |Y| > |Z| (8 ≤ 8), поэтому вызывается mergeAt(arr, stack, 0).

5. merge_runs(arr, 0, 8, 16);

left: [8, 7, -6, 5, 4, 3, 2, -1]

right: [-8, -8, 7, -7, 7, 6, -5, 4]

8 и -8: модули равны - берётся левый 8 (count_left = 1). 7 и -8: берётся -8 из right (count_right = 1), затем ещё раз -8 (count_right = 2). 7 и 7: равны - берётся левый 7 (count_left = 1, count_right = 0). -6 и 7: берётся 7 из right (1), -6 и -7: берётся -7 (2), -6 и 7: берётся 7 (3).

Состояние: arr[0..7): [8, -8, -8, 7, 7, -7, 7]; count_right = 3

count_right дошёл до GALLOP = 3 - вход в галоп, gallops = 1. gallop(right, 5, 8, -6): right[5] = 6, |6| ≥ |-6| - подходит; смещение 1: right[6] = -5, |-5| < 6 - уже нет. Значит, забирается ровно один элемент - 6. Счётчики обнуляются.

Состояние: arr[0..8): [8, -8, -8, 7, 7, -7, 7, 6]

Дальше обычное слияние: -6 и -5 - берётся -6, 5 и -5 - равны, берётся левый 5, 4 и -5 - берётся -5, 4 и 4 - равны, берётся левый 4, 3 и 4 - берётся 4 из right. Правый массив закончился, хвост left - 3, 2, -1 - докопируется без сравнений.

Печатается «Gallops 0: 1» и «Merge 0: 8 -8 -8 7 7 -7 7 6 -6 5 -5 4 4 3 2 -1», а в стеке остаётся одна запись (len 16, start 0).

6. mergeForceCollapse и ответ;

В стеке один ран, сливать больше нечего.

Итоговое состояние: Answer: 8 -8 -8 7 7 -7 7 6 -6 5 -5 4 4 3 2 -1

На этой цепочке хорошо видно, как работает TimSort: возрастающий кусок не сортируется заново, а разворачивается за один проход, невозрастающий берётся как есть, а при слиянии длинная серия побед одной стороны сразу переключает алгоритм в галоп.

Исследование

Чтобы оценить сложность сортировки, был проведён нагрузочный тест. Проверялись такие гипотезы о времени работы:

- линейное время O(n) - в лучшем случае, когда массив уже упорядочен по убыванию модуля;
- время O(n log n) - в среднем случае (случайные данные);
- время O(n log n) - в худшем случае, без квадратичной деградации;
- время заметно зависит от min_run: при слишком маленьком min_run много слияний, при слишком большом - начинают преобладать квадратичные вставки, а оптимум лежит около 32–64.

Каждый случай замерялся отдельно на массивах от 10 до 1 000 000 элементов, на каждый размер делалось несколько повторов, а за результат бралась медиана. Кроме времени, программа считает число сравнений - оно не зависит от загрузки машины и лучше всего показывает асимптотику. Для сравнения на графики нанесены теоретические кривые. Для замеров используется копия алгоритма из src/modules/timsort.h со счётчиками и отключённым выводом. Код нагрузочного теста и построения графиков приведён в приложении А.

Исходные данные, используемые при исследовании:

- тип элементов - целые числа (int), сортировка по убыванию модуля;
- min_run вычисляется стандартным алгоритмом (порог 64), галоп начинается после 7 побед подряд;
- размеры массива n - от 10 до 1 000 000 (16 точек), отдельно в таблице 1 выписаны n = 10, 1000 и 100 000;
- лучший случай - массив, уже отсортированный по убыванию модуля (знаки случайные): весь массив - один ран;
- средний случай - случайные числа от -10^9 до 10^9;
- худший случай - специально построенный массив: внутри каждого блока длины min_run числа идут «пилой» (2, 1, 4, 3, …), поэтому раны имеют длину 2 и вставки делают максимум сдвигов, а значения разложены по блокам через бит-реверсную перестановку, поэтому при

каждом слиянии элементы берутся по очереди то из левого, то из правого рана и галоп не включается;

- дополнительно - частично упорядоченный массив: отсортированный, в котором 1% элементов переставлен случайно;
- число прогревочных прогонов WARMUP = 1, число измеряемых повторов REPEAT = 9, результатом служила медиана; весь прогон повторялся 5 раз, и бралась медиана ещё раз;
- для маленьких n сортировалась пачка из 200 000 / n копий массива, и время делилось на их число;
- измеряемые величины - время одной сортировки в микросекундах и число сравнений (счётчик в функции before).

В таблице 1 приведены результаты для размеров из задания. В столбце «Оценка» стоит n для лучшего случая и $n \cdot \log_2 n$ для среднего и худшего, а в последнем столбце - отношение числа сравнений к этой оценке.

Таблица 1 - Результаты замеров для n = 10, 1000 и 100 000

Случай	n	min_run	Время, мкс	Сравнений	Оценка	Отношение
Лучший	10	10	0,045	19	10	1,90
Лучший	1 000	63	2,118	1 999	1 000	2,00
Лучший	100 000	49	241,833	199 999	100 000	2,00
Средний	10	10	0,054	33	33	0,99
Средний	1 000	63	20,788	20 297	9 966	2,04
Средний	100 000	49	7 374,910	2 400 019	1 660 964	1,44
Худший	10	10	0,067	48	33	1,44
Худший	1 000	63	30,869	34 814	9 966	3,49
Худший	100 000	49	3 981,028	3 503 764	1 660 964	2,11

Ниже на рисунках 1-4 приведены результаты замеров для всех размеров массива.

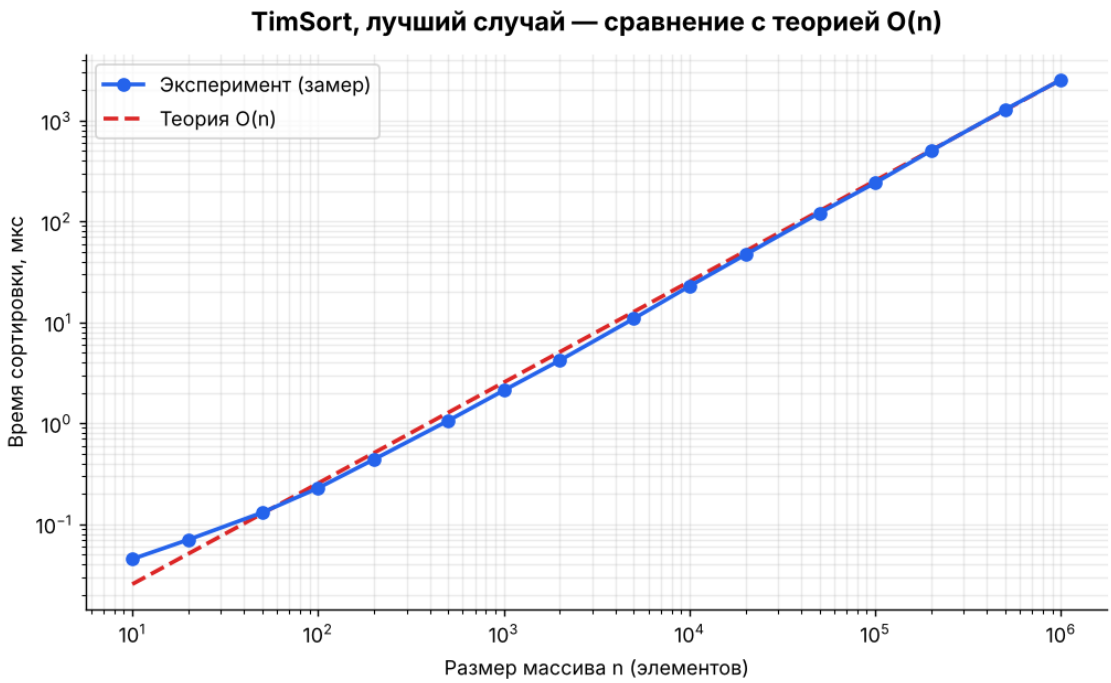


Рисунок 1 - Время работы в лучшем случае (сравнение с O(n))

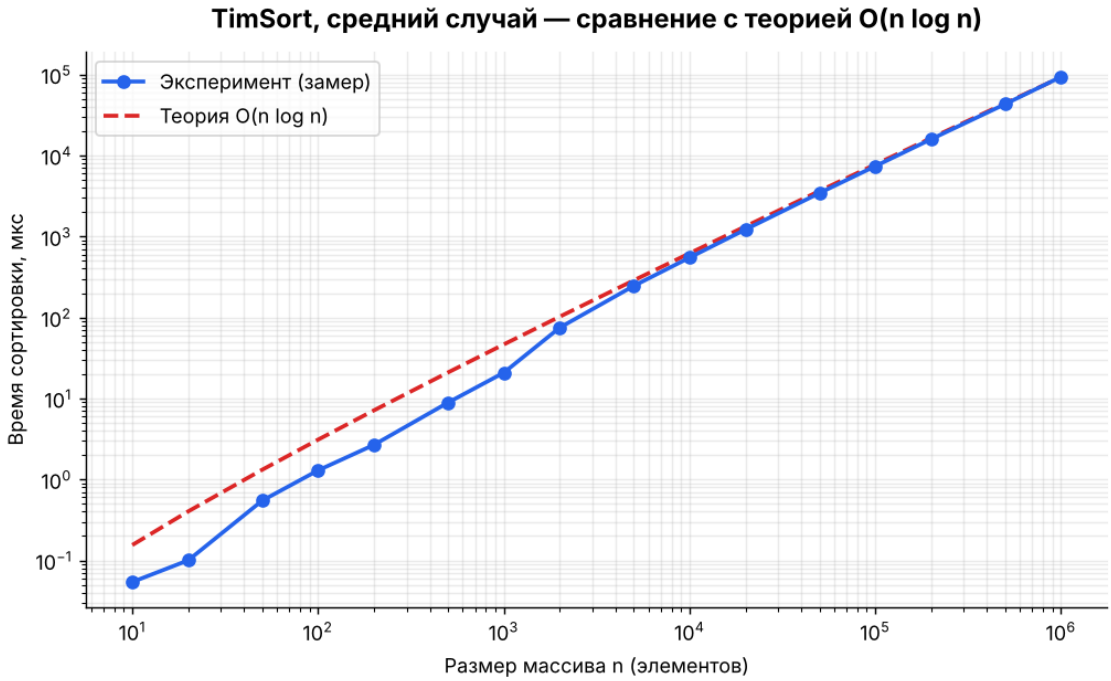


Рисунок 2 - Время работы в среднем случае (сравнение с O(n log n))

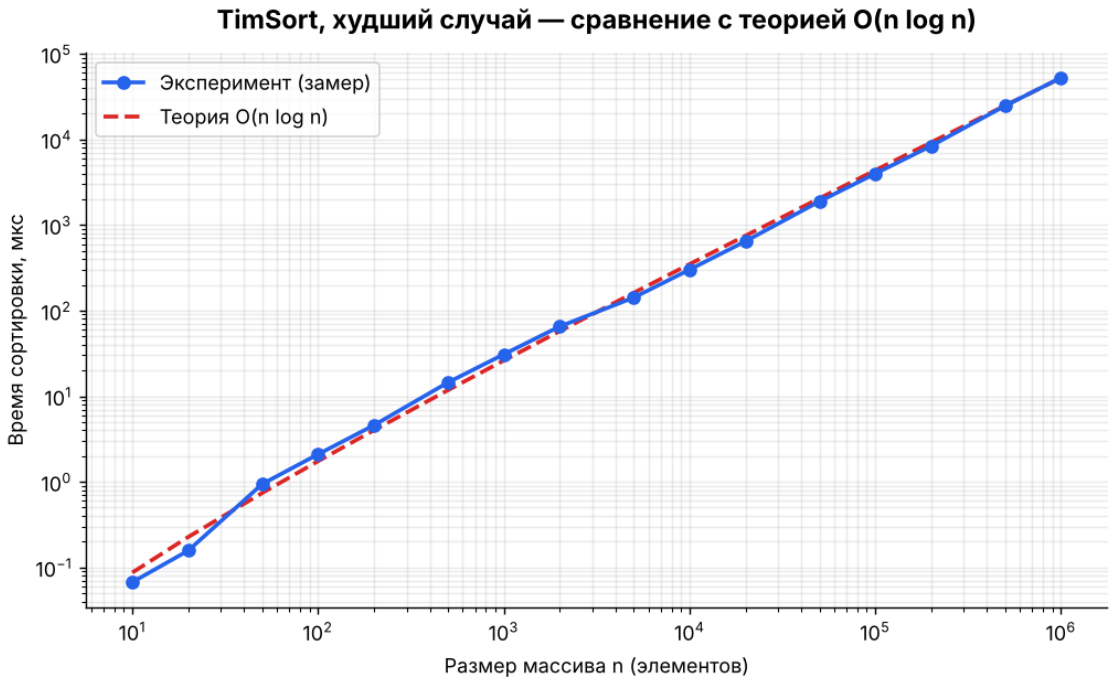


Рисунок 3 - Время работы в худшем случае (сравнение с O(n log n))

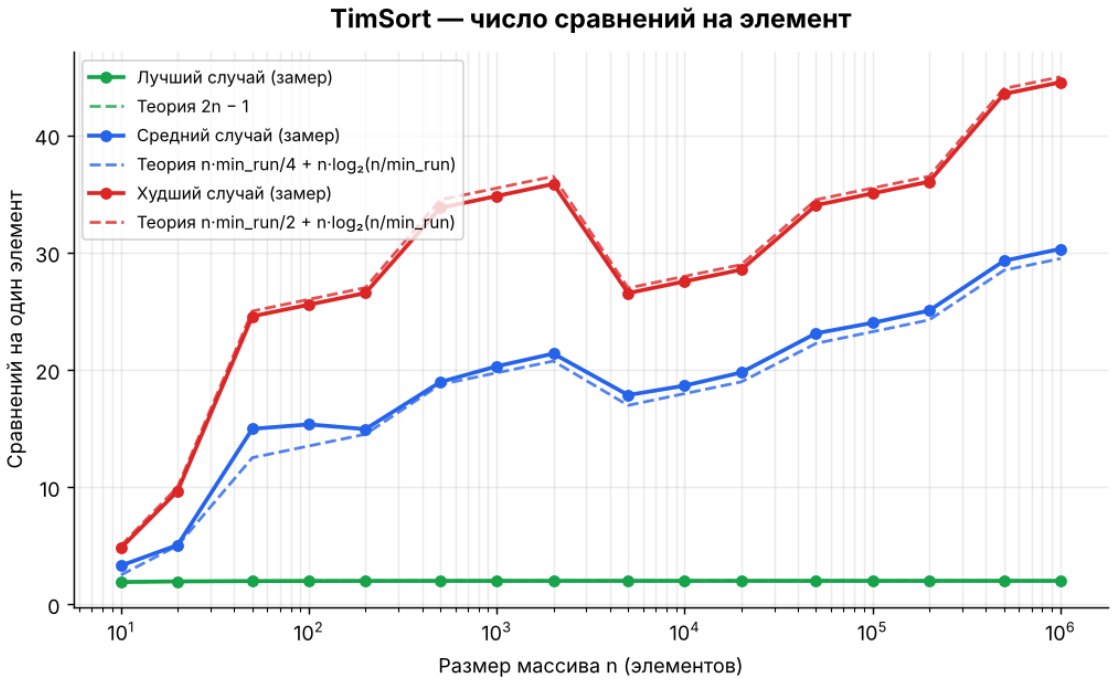


Рисунок 4 - Число сравнений на один элемент для трёх случаев

Основные наблюдения

- Лучший случай линейный: на уже упорядоченном массиве делается ровно 2n - 1 сравнение - n - 1 в find_run и ещё n - 1, когда insertion_sort проходит по готовому рану, ничего не сдвигая. Время растёт пропорционально n: 0,045 мкс при n = 10, 2,12 мкс при n = 1000 и 241,8 мкс

при n = 100 000, а на рисунке 1 точки ложатся на прямую O(n). Половину этих сравнений можно сэкономить, если не вызывать сортировку вставками для естественного рана длиннее min_run.

- Средний случай - O(n log n). Отношение числа сравнений к $n \cdot \log_2 n$ не постоянно (0,99, 2,04 и 1,44), потому что вместе с n меняется и min_run (10, 63 и 49), а вставки дают вклад около n·min_run/4. Если учесть это, модель $n \cdot \text{min_run}/4 + n \cdot \log_2(n/\text{min_run})$ совпадает с замерами с точностью около 3%: для n = 100 000 она даёт 2 324 493 сравнений против 2 400 019 измеренных. Волны на рисунке 4 - это как раз скачки min_run при переходе n через степени двойки.
- Худший случай тоже O(n log n): сравнений больше, чем в среднем, примерно в 1,46 раза при n = 100 000, но квадратичного роста нет - отношение к $n \cdot \log_2 n$ держится около 2, а модель $n \cdot \text{min_run}/2 + n \cdot \log_2(n/\text{min_run})$ расходится с замером меньше чем на 2%. Это и обеспечивает инвариант стека: как бы ни были устроены данные, сливаются куски соизмеримой длины.
- При этом по времени худший случай оказался быстрее среднего: 3 981 мкс против 7 375 мкс при n = 100 000. Дело в предсказании переходов процессора: на случайных данных ветка «взять слева или справа» непредсказуема и процессор часто ошибается, а в построенном худшем случае элементы строго чередуются, и переходы угадываются. Поэтому худший случай по числу сравнений не обязательно худший по времени.
- Частично упорядоченный массив сортируется в 7,6 раза быстрее случайного (966 мкс против 7 375 мкс при n = 100 000): длинные раны и галоп почти полностью избавляют от сравнений.
- Всплеск на маленьких n: при $n \leq 1000$ время растёт быстрее теоретической кривой - на маленьких массивах сортировка сводится к вставкам, а на больших данные перестают помещаться в кэш. Начиная с n около 10 000 точки идут вдоль теоретической прямой: при переходе от 100

000 к 1 000 000 время выросло в 12,7 раза при теоретическом росте $n \cdot \log_2 n$ в 12 раз.

Влияние min_run

Чтобы проверить, как min_run влияет на скорость, он задавался вручную (FIXED_MINRUN) от 1 до 2048, а сортировались случайные массивы из 1000 и 100 000 элементов. Результаты сведены в таблицу 2 и на рисунки 5-7.

Таблица 2 - Зависимость времени и числа сравнений от min_run (случайные данные)

min_run	Время, мкс (n = 1000)	Сравнений (n = 1000)	Время, мкс (n = 100 000)	Сравнений (n = 100 000)	Слияний (n = 100 000)
1	50,21	10 539	10 349	1 731 806	41 350
4	40,52	9 764	9 286	1 724 467	24 888
8	29,77	9 642	8 541	1 648 406	12 499
16	20,46	10 516	7 954	1 751 487	6 249
32	19,19	13 525	7 552	2 054 851	3 124
48	19,16	17 616	7 488	2 398 978	2 083
64	21,02	20 147	7 317	2 756 024	1 562
128	29,33	35 026	7 633	4 251 206	781
256	44,96	65 714	8 792	7 344 209	390
512	76,54	127 289	11 903	13 622 954	195
1024	-	-	17 773	26 267 212	97
2048	-	-	31 004	51 763 556	48

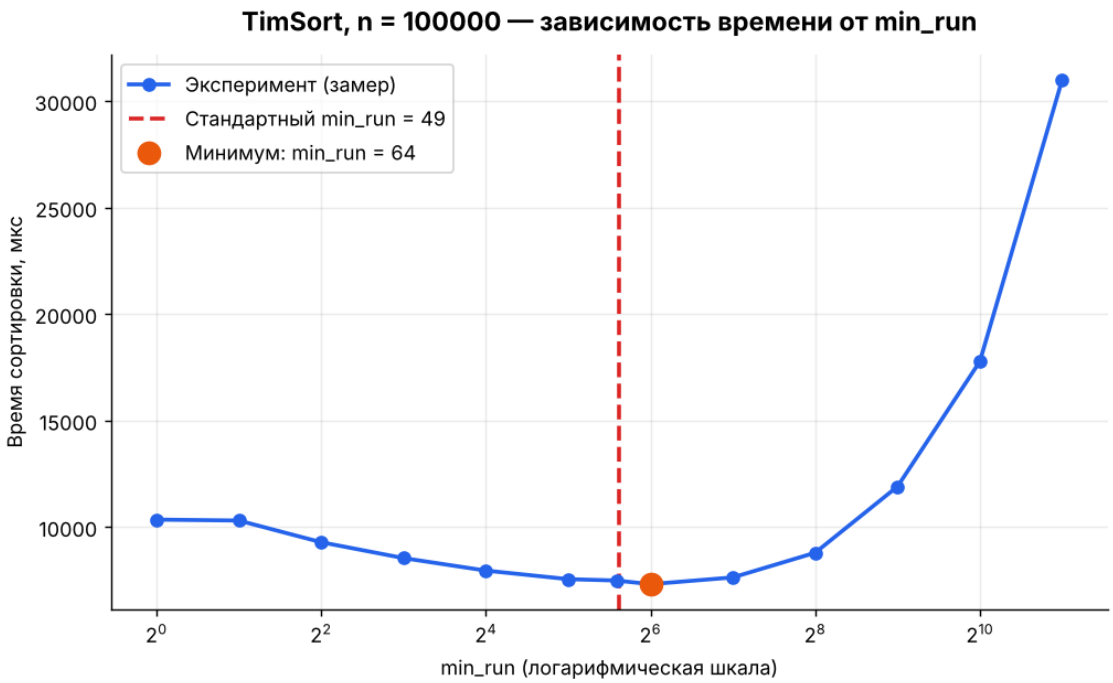


Рисунок 5 - Зависимость времени от min_run при n = 100 000

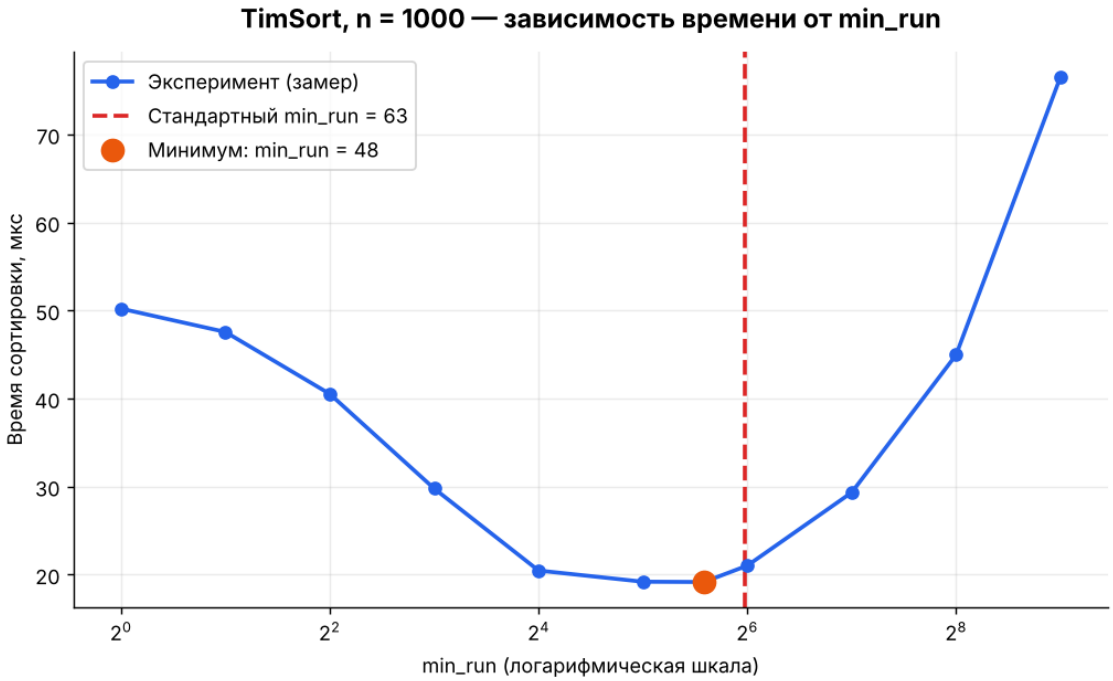


Рисунок 6 - Зависимость времени от min_run при n = 1000

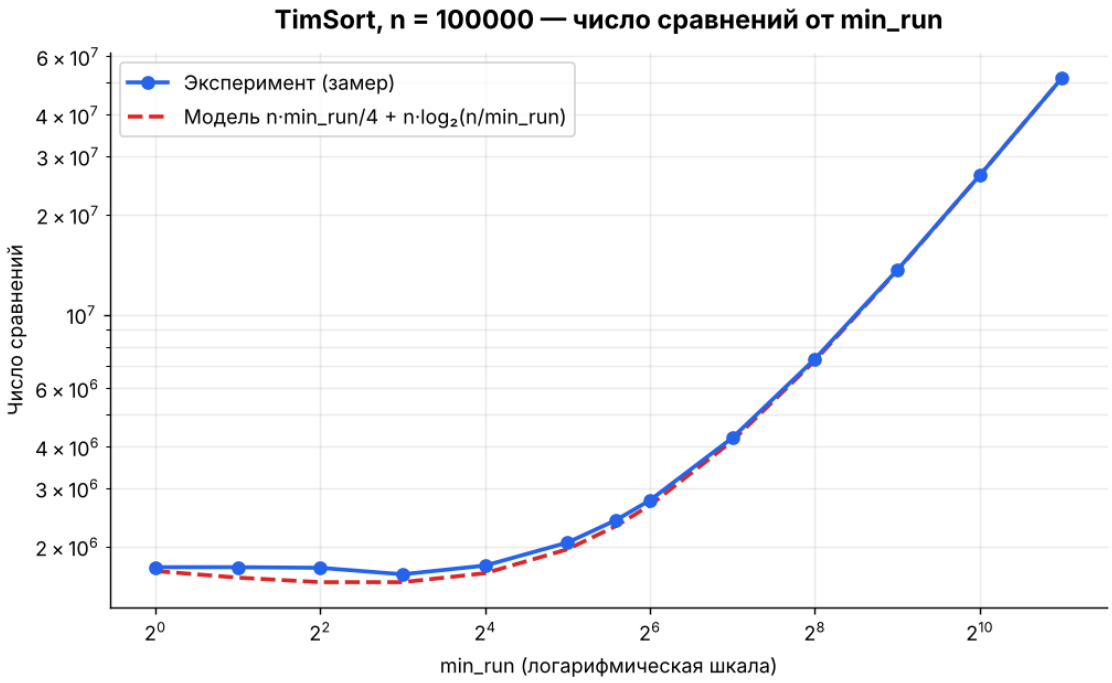


Рисунок 7 - Число сравнений в зависимости от min_run при n = 100 000

- Зависимость времени от min_run имеет форму буквы U (рисунки 5 и 6). При n = 100 000 при min_run = 1 сортировка идёт 10 349 мкс - вставок почти нет, зато 41 350 слияний, и TimSort превращается в обычную сортировку слиянием с накладными расходами на стек и копирование. При min_run = 2048 время вырастает до 31 004 мкс - начинают преобладать квадратичные вставки.
- Оптимум лежит в районе 32–64: при n = 100 000 минимум 7 317 мкс достигается при min_run = 64, а стандартный min_run = 49 (в таблице - 48) даёт 7 488 мкс, всего на 2% больше. При n = 1000 минимум приходится на 32–48 (19,19 и 19,16 мкс), а стандартный min_run = 63 даёт 20,79 мкс.
- По числу сравнений оптимум другой - около min_run = 8 (1 648 406 сравнений), а при 64 сравнений уже 2 756 024. Но сравнение во вставках дешевле, чем в слиянии: вставки идут по соседним ячейкам памяти, а каждое слияние копирует данные во временные векторы. Поэтому по времени выгоднее min_run побольше, и стандартный алгоритм выбирает именно 32–64. На рисунке 7 видно, что модель $n \cdot min_run/4 + n \cdot \log_2(n/min_run)$ хорошо описывает число сравнений при любом min_run.

Влияние галопа

Отдельно проверено, что даёт режим галопа: тот же массив из 100 000 элементов сортировался с галопом после 7 побед и с отключённым галопом (порог больше длины массива).

Таблица 3 - Влияние галопа при n = 100 000

Данные	Галоп	Время, мкс	Сравнений	Входов в галоп
Случайные	после 7 побед	7 532	2 404 090	8 436
Случайные	отключён	7 420	2 394 584	0
Частично упорядоченные	после 7 побед	945	312 333	4 493
Частично упорядоченные	отключён	2 311	1 257 078	0

На случайных данных галоп почти ничего не меняет: серии из 7 побед подряд редки, и время отличается в пределах погрешности (7 532 и 7 420 мкс). Зато на частично упорядоченных данных галоп сокращает число сравнений в 4,0 раза и ускоряет сортировку в 2,4 раза - длинные куски забираются из одного рана целиком за O(log k).

Из замеров видно, что TimSort даёт:

- линейное время O(n) на уже упорядоченных данных;
- O(n log n) в среднем и худшем случаях - без квадратичной деградации, которую могла бы дать сортировка вставками;
- сильное ускорение на частично упорядоченных данных за счёт естественных ранов и галопа;
- лучшее время при min_run около 32–64 - именно в этот диапазон попадает стандартный алгоритм вычисления min_run.

В целом реализация ведёт себя так, как и предсказывает теория.

Вывод

В работе разобрано, как устроен и как работает гибридный алгоритм сортировки TimSort: поиск естественных ранов с разворотом убывающих кусков, вычисление min_run, сортировка вставками коротких блоков, слияние со стеком ранов и режим галопа.

На C++ написана сортировка массива по убыванию модуля, которая закрывает все требования задания: печатает разделённые блоки, число входов в галоп и результат каждого слияния. Сравнение элементов при вставках и слиянии собрано в функции before, инвариант стека поддерживается вместе с исправлением 2015 года, а галоп сделан через экспоненциальный и бинарный поиск. Вывод программы совпадает с эталоном на всех тестах тестирующей системы, а 15 тестов на Google Test, подключающих сам main.cpp, проверяют отдельные функции, примеры из условия, граничные случаи и большие случайные массивы.

Исследование подтвердило теоретические оценки: на упорядоченных данных сортировка линейная (ровно 2n - 1 сравнение), в среднем и худшем случаях - O(n log n), причём число сравнений с точностью в несколько процентов описывается моделью $n \cdot min_run/4 + n \cdot \log_2(n/min_run)$. Зависимость времени от min_run имеет форму буквы U с минимумом около 32–64, а галоп почти не влияет на случайные данные, но в 2,4 раза ускоряет сортировку частично упорядоченных.

Отдельно выяснилось, что для совпадения с эталоном галоп по правому рану должен забирать элементы с равным модулем, из-за чего на таких элементах сортировка теряет устойчивость; все остальные шаги алгоритма устойчивы.

ПРИЛОЖЕНИЕ А

Исходный код программы

Файл src/main.cpp

```
#include <iostream>
#include <vector>
#include <cmath>

using namespace std;
const int MINRUN_LIMIT = 16;
const int GALLOP = 3;
int mergeCounter = 0;
struct Run {
    int len;
    int start;
};
void printRange(const vector<int>& a, int lo, int hi) {
    for (int i = lo; i < hi; i++) {
        if (i > lo) cout << " ";
        cout << a[i];
    }
    cout << "\n";
}
bool before(int a, int b) {
    return abs(a) > abs(b);
}
int calc_minrun(int n) {
    int r = 0;
    while (n >= MINRUN_LIMIT) {
        r |= n & 1;
        n >>= 1;
    }
    return n + r;
}
void reverseRange(vector<int>& arr, int lo, int hi) {
    hi--;
    while (lo < hi) {
        int t = arr[lo];
        arr[lo] = arr[hi];
        arr[hi] = t;
        hi--;
        lo++;
    }
}
int find_run(vector<int>& arr, int lo, int n) {
    int hi = lo + 1;
    if (hi == n) return hi;
    if (abs(arr[lo]) < abs(arr[hi])) {
        while (hi < n && abs(arr[hi]) > abs(arr[hi - 1])) hi++;
        reverseRange(arr, lo, hi);
    } else {
        while (hi < n && abs(arr[hi]) <= abs(arr[hi - 1])) hi++;
    }
    return hi;
}
void insertion_sort(vector<int>& arr, int lo, int hi) {
    for (int i = lo; i < hi; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= lo && before(key, arr[j])) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
bool goesFirst(int x, int key) {
```

```
    return abs(x) >= abs(key);
}
int gallop(const vector<int>& arr, int from, int end, int key) {
    int len = end - from;
    if (len <= 0 || !goesFirst(arr[from], key)) return 0;
    int last_ok = 0;
    int bound = 1;
    while (bound < len && goesFirst(arr[from + bound], key)) {
        last_ok = bound;
        bound = bound * 2 + 1;
    }
    if (bound > len) bound = len;
    int lo = last_ok + 1;
    int hi = bound;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (goesFirst(arr[from + mid], key)) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
int merge_runs(vector<int>& arr, int lo, int mid, int hi) {
    vector<int> left(arr.begin() + lo, arr.begin() + mid);
    vector<int> right(arr.begin() + mid, arr.begin() + hi);
    int n_left = left.size();
    int n_right = right.size();
    int i = 0, j = 0, k = lo;
    int count_left = 0, count_right = 0, gallops = 0;
    while (i < n_left && j < n_right) {
        if (!before(right[j], left[i])) {
            arr[k++] = left[i++];
            count_left++;
            count_right = 0;
        } else {
            arr[k++] = right[j++];
            count_right++;
            count_left = 0;
        }
        if (count_left == GALLOP) {
            gallops++;
            if (i < n_left && j < n_right) {
                int cnt = gallop(left, i, n_left, right[j]);
                for (int t = 0; t < cnt; t++) arr[k++] = left[i++];
            }
            count_left = 0;
            count_right = 0;
        } else if (count_right == GALLOP) {
            gallops++;
            if (i < n_left && j < n_right) {
                int cnt = gallop(right, j, n_right, left[i]);
                for (int t = 0; t < cnt; t++) arr[k++] = right[j++];
            }
            count_left = 0;
            count_right = 0;
        }
    }
    while (i < n_left) arr[k++] = left[i++];
    while (j < n_right) arr[k++] = right[j++];
    return gallops;
}
void mergeAt(vector<int>& arr, vector<Run>& stack, int k) {
    int lo = stack[k].start;
    int mid = lo + stack[k].len;
    int hi = mid + stack[k + 1].len;
    int gallops = merge_runs(arr, lo, mid, hi);
    cout << "Gallops " << mergeCounter << ": " << gallops << "\n";
    cout << "Merge " << mergeCounter << ": ";
    printRange(arr, lo, hi);
    mergeCounter++;
    stack[k].len += stack[k + 1].len;
    stack.erase(stack.begin() + k + 1);
```

```
}
void merge_collapse(vector<int>& arr, vector<Run>& stack) {
    while (stack.size() > 1) {
        int k = stack.size() - 2;
        if ((k > 0 && stack[k - 1].len <= stack[k].len + stack[k + 1].len) ||
            (k > 1 && stack[k - 2].len <= stack[k - 1].len + stack[k].len)) {
            if (stack[k - 1].len < stack[k + 1].len) k--;
            mergeAt(arr, stack, k);
        } else if (stack[k].len <= stack[k + 1].len) {
            mergeAt(arr, stack, k);
        } else {
            break;
        }
    }
}
void mergeForceCollapse(vector<int>& arr, vector<Run>& stack) {
    while (stack.size() > 1) {
        int k = stack.size() - 2;
        if (k > 0 && stack[k - 1].len < stack[k + 1].len) k--;
        mergeAt(arr, stack, k);
    }
}
void tim_sort(vector<int>& arr, int n) {
    int min_run = calc_minrun(n);
    int lo = 0;
    int part = 0;
    vector<Run> runs;
    while (lo < n) {
        int hi = find_run(arr, lo, n);
        if (hi - lo < min_run) hi = min(lo + min_run, n);
        insertion_sort(arr, lo, hi);
        cout << "Part " << part << ": ";
        printRange(arr, lo, hi);
        part++;
        runs.push_back({hi - lo, lo});
        lo = hi;
    }
    vector<Run> stack;
    for (int r = 0; r < (int)runs.size(); r++) {
        stack.push_back(runs[r]);
        merge_collapse(arr, stack);
    }
    mergeForceCollapse(arr, stack);
    cout << "Answer: ";
    printRange(arr, 0, n);
}
int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];
    tim_sort(arr, n);
    return 0;
}
```

Файл src/modules/timsort.h

```
#pragma once
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

using namespace std;

inline int MINRUN_LIMIT = 16;
inline int GALLOP = 3;
inline int FIXED_MINRUN = 0;
inline bool PRINT = true;
inline long long comparisons = 0;
inline long long gallopsTotal = 0;
```

```
inline int mergeCounter = 0;

struct Run {
    int len;
    int start;
};

inline void printRange(const vector<int>& a, int lo, int hi) {
    for (int i = lo; i < hi; i++) {
        if (i > lo) cout << " ";
        cout << a[i];
    }
    cout << "\n";
}

inline bool before(int a, int b) {
    comparisons++;
    return abs(a) > abs(b);
}

inline int calc_minrun(int n) {
    if (FIXED_MINRUN > 0) return FIXED_MINRUN;
    int r = 0;
    while (n >= MINRUN_LIMIT) {
        r |= n & 1;
        n >>= 1;
    }
    return n + r;
}

inline void reverseRange(vector<int>& arr, int lo, int hi) {
    hi--;
    while (lo < hi) {
        int t = arr[lo];
        arr[lo] = arr[hi];
        arr[hi] = t;
        hi--;
        lo++;
    }
}

inline int find_run(vector<int>& arr, int lo, int n) {
    int hi = lo + 1;
    if (hi == n) return hi;
    if (before(arr[hi], arr[lo])) {
        while (hi < n && before(arr[hi], arr[hi - 1])) hi++;
        reverseRange(arr, lo, hi);
    } else {
        while (hi < n && !before(arr[hi], arr[hi - 1])) hi++;
    }
    return hi;
}

inline void insertion_sort(vector<int>& arr, int lo, int hi) {
    for (int i = lo; i < hi; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= lo && before(key, arr[j])) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

inline bool goesFirst(int x, int key) {
    return !before(key, x);
}

inline int gallop(const vector<int>& arr, int from, int end, int key) {
    int len = end - from;
```

```
    if (len <= 0 || !goesFirst(arr[from], key)) return 0;
    int last_ok = 0;
    int bound = 1;
    while (bound < len && goesFirst(arr[from + bound], key)) {
        last_ok = bound;
        bound = bound * 2 + 1;
    }
    if (bound > len) bound = len;
    int lo = last_ok + 1;
    int hi = bound;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (goesFirst(arr[from + mid], key)) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

inline int merge_runs(vector<int>& arr, int lo, int mid, int hi) {
    vector<int> left(arr.begin() + lo, arr.begin() + mid);
    vector<int> right(arr.begin() + mid, arr.begin() + hi);
    int n_left = left.size();
    int n_right = right.size();
    int i = 0, j = 0, k = lo;
    int count_left = 0, count_right = 0, gallops = 0;
    while (i < n_left && j < n_right) {
        if (!before(right[j], left[i])) {
            arr[k++] = left[i++];
            count_left++;
            count_right = 0;
        } else {
            arr[k++] = right[j++];
            count_right++;
            count_left = 0;
        }
        if (count_left == GALLOP) {
            gallops++;
            if (i < n_left && j < n_right) {
                int cnt = gallop(left, i, n_left, right[j]);
                for (int t = 0; t < cnt; t++) arr[k++] = left[i++];
            }
            count_left = 0;
            count_right = 0;
        } else if (count_right == GALLOP) {
            gallops++;
            if (i < n_left && j < n_right) {
                int cnt = gallop(right, j, n_right, left[i]);
                for (int t = 0; t < cnt; t++) arr[k++] = right[j++];
            }
            count_left = 0;
            count_right = 0;
        }
    }
    while (i < n_left) arr[k++] = left[i++];
    while (j < n_right) arr[k++] = right[j++];
    return gallops;
}

inline void mergeAt(vector<int>& arr, vector<Run>& stack, int k) {
    int lo = stack[k].start;
    int mid = lo + stack[k].len;
    int hi = mid + stack[k + 1].len;
    int gallops = merge_runs(arr, lo, mid, hi);
    gallopsTotal += gallops;
    if (PRINT) {
        cout << "Gallops " << mergeCounter << ": " << gallops << "\n";
        cout << "Merge " << mergeCounter << ": ";
        printRange(arr, lo, hi);
    }
    mergeCounter++;
    stack[k].len += stack[k + 1].len;
```

```
    stack.erase(stack.begin() + k + 1);
}

inline void merge_collapse(vector<int>& arr, vector<Run>& stack) {
    while (stack.size() > 1) {
        int k = stack.size() - 2;
        if ((k > 0 && stack[k - 1].len <= stack[k].len + stack[k + 1].len) ||
            (k > 1 && stack[k - 2].len <= stack[k - 1].len + stack[k].len)) {
            if (stack[k - 1].len < stack[k + 1].len) k--;
            mergeAt(arr, stack, k);
        } else if (stack[k].len <= stack[k + 1].len) {
            mergeAt(arr, stack, k);
        } else {
            break;
        }
    }
}

inline void mergeForceCollapse(vector<int>& arr, vector<Run>& stack) {
    while (stack.size() > 1) {
        int k = stack.size() - 2;
        if (k > 0 && stack[k - 1].len < stack[k + 1].len) k--;
        mergeAt(arr, stack, k);
    }
}

inline void tim_sort(vector<int>& arr, int n) {
    mergeCounter = 0;
    int min_run = calc_minrun(n);
    int lo = 0;
    int part = 0;
    vector<Run> runs;
    while (lo < n) {
        int hi = find_run(arr, lo, n);
        if (hi - lo < min_run) hi = min(lo + min_run, n);
        insertion_sort(arr, lo, hi);
        if (PRINT) {
            cout << "Part " << part << ": ";
            printRange(arr, lo, hi);
        }
        part++;
        runs.push_back({hi - lo, lo});
        lo = hi;
    }
    vector<Run> stack;
    for (int r = 0; r < (int)runs.size(); r++) {
        stack.push_back(runs[r]);
        merge_collapse(arr, stack);
    }
    mergeForceCollapse(arr, stack);
    if (PRINT) {
        cout << "Answer: ";
        printRange(arr, 0, n);
    }
}
```

Файл src/modules/tests.cpp

```
#include <gtest/gtest.h>
#include <random>
#include <sstream>
#include <string>
#include <vector>

#define main timsort_main
#include "../main.cpp"
#undef main

static string run_sort(vector<int> arr) {
    mergeCounter = 0;
    testing::internal::CaptureStdout();
```

```
    tim_sort(arr, arr.size());
    return testing::internal::GetCapturedStdout();
}

TEST(TimSort, CalcMinRun) {
    EXPECT_EQ(calc_minrun(20), 10);
    EXPECT_EQ(calc_minrun(16), 8);
    EXPECT_EQ(calc_minrun(15), 15);
    EXPECT_EQ(calc_minrun(33), 9);
    EXPECT_EQ(calc_minrun(1000), 16);
}

TEST(TimSort, FindRunReversesAscending) {
    vector<int> a = {1, -2, 3, -4, 5, 6, -7, -8, 9, -10, 11, -10, -9};
    EXPECT_EQ(find_run(a, 0, a.size()), 11);
    vector<int> expected = {11, -10, 9, -8, -7, 6, 5, -4, 3, -2, 1, -10, -9};
    EXPECT_EQ(a, expected);
}

TEST(TimSort, FindRunKeepsNonIncreasing) {
    vector<int> a = {5, -5, 5, 4, 7};
    EXPECT_EQ(find_run(a, 0, a.size()), 4);
    vector<int> expected = {5, -5, 5, 4, 7};
    EXPECT_EQ(a, expected);
}

TEST(TimSort, InsertionSortIsStable) {
    vector<int> a = {1, -3, 2, 3, -1, -2};
    insertion_sort(a, 0, a.size());
    vector<int> expected = {-3, 3, 2, -2, 1, -1};
    EXPECT_EQ(a, expected);
}

TEST(TimSort, GallopFindsBlockLength) {
    vector<int> a = {9, 8, 7, -7, 5, 3};
    EXPECT_EQ(gallop(a, 0, a.size(), 7), 4);
    EXPECT_EQ(gallop(a, 0, a.size(), 10), 0);
    EXPECT_EQ(gallop(a, 0, a.size(), 1), 6);
}

TEST(TimSort, ExampleOneFromTask) {
    string out = run_sort({1, -2, 3, -4, 5, 6, -7, -8, 9, -10, 11, -10, -9, 8, 7, -
7, -6, 6, 5, 4});
    EXPECT_EQ(out,
        "Part 0: 11 -10 9 -8 -7 6 5 -4 3 -2 1\n"
        "Part 1: -10 -9 8 7 -7 -6 6 5 4\n"
        "Gallops 0: 0\n"
        "Merge 0: 11 -10 -10 9 -9 -8 8 -7 7 -7 6 -6 6 5 5 -4 4 3 -2 1\n"
        "Answer: 11 -10 -10 9 -9 -8 8 -7 7 -7 6 -6 6 5 5 -4 4 3 -2 1\n");
}

TEST(TimSort, ExampleTwoFromTask) {
    string out = run_sort({-1, 2, 3, 4, 5, -6, 7, 8, -8, -8, 7, -7, 7, 6, -5, 4});
    EXPECT_EQ(out,
        "Part 0: 8 7 -6 5 4 3 2 -1\n"
        "Part 1: -8 -8 7 -7 7 6 -5 4\n"
        "Gallops 0: 1\n"
        "Merge 0: 8 -8 -8 7 7 -7 7 6 -6 5 -5 4 4 3 2 -1\n"
        "Answer: 8 -8 -8 7 7 -7 7 6 -6 5 -5 4 4 3 2 -1\n");
}

TEST(TimSort, GallopCountedWhenRunEnds) {
    string out = run_sort({3, -3, 2, 2, -2, 1, 1, -1, 100, -10, -5, -4, 3, 3, 2,
1});
    EXPECT_NE(out.find("Gallops 0: 3\n"), string::npos);
    EXPECT_NE(out.find("Answer: 100 -10 -5 -4 3 3 3 -3 2 2 -2 2 1 1 -1 1\n"),
string::npos);
}

TEST(TimSort, GallopOnEqualModules) {
    string out = run_sort({10, -10, 10, -10, -10, -10, 10, 10, 11, 10, -10, 10, -
```

```
10, -10, 10, -10});
    EXPECT_NE(out.find("Gallops 0: 1\n"), string::npos);
    EXPECT_NE(out.find("Answer: 11 10 -10 10 -10 -10 -10 10 10 10 -10 10 -10 -10 10
-10\n"), string::npos);
}

TEST(TimSort, FourBlocksStackInvariant) {
    string out = run_sort({10, -10, 10, -10, -10, -10, 10, 10, 10, 11, 10, -10, 10,
-10, -10, 10, -10,
                           25, -25, -15, 13, -12, -11, 11, -11, 15, -13, 12, 11, -
11, 11, 10});
    EXPECT_NE(out.find("Part 3: 15 -13 12 11 -11 11 10\nGallops 0: 1\n"),
string::npos);
    EXPECT_NE(out.find("Merge 0: 25 -25 -15 13 -12 -11 11 -11 11 10 -10 10 -10 -10
10 -10\n"), string::npos);
    EXPECT_NE(out.find("Gallops 2: 3\n"), string::npos);
}

TEST(TimSort, SevenGallopsInLastMerge) {
    string out = run_sort({-9, -8, 7, 5, 5, -4, -4, 3, -3, 2, -1, -10, 9, -7, 6, -
4, -4, -3, 2, -2, 1, 0,
                           -8, -8, -8, -8, 8, -7, 5, -3, -1, -1, 0, -9, 9, -8, 6, -
6, -3, 3, 3, -1, 1, -1});
    EXPECT_NE(out.find("Gallops 0: 1\n"), string::npos);
    EXPECT_NE(out.find("Gallops 1: 2\n"), string::npos);
    EXPECT_NE(out.find("Gallops 2: 7\n"), string::npos);
}

TEST(TimSort, SingleElement) {
    EXPECT_EQ(run_sort({-43}), "Part 0: -43\nAnswer: -43\n");
}

TEST(TimSort, AllEqualModules) {
    EXPECT_EQ(run_sort({10, -10, 10}), "Part 0: 10 -10 10\nAnswer: 10 -10 10\n");
}

TEST(TimSort, AlreadySortedOneRun) {
    EXPECT_EQ(run_sort({-6, -6, -6, 6, 5, -5, 5, 4, -4, 4}),
              "Part 0: -6 -6 -6 6 5 -5 5 4 -4 4\nAnswer: -6 -6 -6 6 5 -5 5 4 -4
4\n");
}

TEST(TimSort, RandomArraysSortedByModule) {
    mt19937 rng(42);
    for (int iter = 0; iter < 200; iter++) {
        int n = 1 + rng() % 2000;
        vector<int> a(n);
        for (int i = 0; i < n; i++) a[i] = (int)(rng() % 41) - 20;
        vector<int> src = a;
        mergeCounter = 0;
        testing::internal::CaptureStdout();
        tim_sort(a, n);
        testing::internal::GetCapturedStdout();
        for (int i = 1; i < n; i++) ASSERT_GE(abs(a[i - 1]), abs(a[i]));
        vector<int> x = a, y = src;
        sort(x.begin(), x.end());
        sort(y.begin(), y.end());
        ASSERT_EQ(x, y);
    }
}
```

Файл src/modules/benchmark.cpp

```
#include <algorithm>
#include <chrono>
#include <cstdio>
#include <random>
#include <string>
#include <vector>

#include "timsort.h"
```

```
using namespace std;

static const int WARMUP = 1;
static const int REPEAT = 9;
static mt19937 rng(2026);

static vector<int> gen_best(int n) {
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = (rng() & 1 ? 1 : -1) * (n - i);
    return a;
}

static vector<int> gen_average(int n) {
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = (int)(rng() % 2000000001u) - 1000000000;
    return a;
}

static unsigned bit_reverse(unsigned x, int bits) {
    unsigned r = 0;
    for (int i = 0; i < bits; i++) {
        r = (r << 1) | (x & 1);
        x >>= 1;
    }
    return r;
}

static vector<int> gen_worst(int n) {
    int min_run = calc_minrun(n);
    int m = (n + min_run - 1) / min_run;
    int bits = 0;
    while ((1 << bits) < m) bits++;
    vector<int> order(m);
    for (int q = 0; q < m; q++) order[q] = q;
    sort(order.begin(), order.end(), [&](int x, int y) { return bit_reverse(x,
bits) < bit_reverse(y, bits); });
    vector<vector<int>> blocks(m);
    int rank = 1;
    for (int round = 0; rank <= n; round++) {
        for (int q : order) {
            int size = min(min_run, n - q * min_run);
            if (round < size) blocks[q].push_back(rank++);
        }
    }
    vector<int> a;
    for (auto& b : blocks) {
        for (size_t i = 0; i + 1 < b.size(); i += 2) swap(b[i], b[i + 1]);
        a.insert(a.end(), b.begin(), b.end());
    }
    return a;
}

static vector<int> gen_partial(int n) {
    vector<int> a = gen_best(n);
    int swaps = max(1, n / 100);
    for (int s = 0; s < swaps; s++) swap(a[rng() % n], a[rng() % n]);
    return a;
}

static vector<int> generate(const string& kind, int n) {
    if (kind == "best") return gen_best(n);
    if (kind == "average") return gen_average(n);
    if (kind == "worst") return gen_worst(n);
    return gen_partial(n);
}

struct Result {
    double time_ns;
    long long comparisons;
    long long gallops;
```

```
    int merges;
};

static Result measure(const vector<int>& src) {
    int n = src.size();
    int batch = max(1, 200000 / n);
    vector<double> times;
    Result res{0, 0, 0, 0};
    for (int rep = 0; rep < WARMUP + REPEAT; rep++) {
        vector<vector<int>> copies(batch, src);
        comparisons = 0;
        gallopsTotal = 0;
        auto t0 = chrono::steady_clock::now();
        for (int b = 0; b < batch; b++) tim_sort(copies[b], n);
        auto t1 = chrono::steady_clock::now();
        if (rep >= WARMUP) times.push_back(chrono::duration<double, nano>(t1 -
t0).count() / batch);
        res.comparisons = comparisons / batch;
        res.gallops = gallopsTotal / batch;
        res.merges = mergeCounter;
    }
    sort(times.begin(), times.end());
    res.time_ns = times[times.size() / 2];
    return res;
}

int main() {
    PRINT = false;
    MINRUN_LIMIT = 64;
    GALLOP = 7;

    printf("experiment,case,n,min_run,gallop,time_ns,comparisons,gal-
lops,merges\n");

    vector<int> sizes = {10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000,
                         20000, 50000, 100000, 200000, 500000, 1000000};
    vector<string> kinds = {"best", "average", "worst", "partial"};
    for (const string& kind : kinds) {
        for (int n : sizes) {
            FIXED_MINRUN = 0;
            vector<int> src = generate(kind, n);
            Result r = measure(src);
            printf("size,%s,%d,%d,%d,%.1f,%lld,%lld,%d\n", kind.c_str(), n,
calc_minrun(n), GALLOP,
                   r.time_ns, r.comparisons, r.gallops, r.merges);
            fflush(stdout);
        }
    }

    vector<int> min_runs = {1, 2, 4, 8, 16, 32, 48, 64, 128, 256, 512, 1024, 2048};
    for (int n : {1000, 100000}) {
        vector<int> src = gen_average(n);
        for (int mr : min_runs) {
            if (mr > n) continue;
            FIXED_MINRUN = mr;
            Result r = measure(src);
            printf("minrun,average,%d,%d,%d,%.1f,%lld,%lld,%d\n", n, mr, GALLOP,
                   r.time_ns, r.comparisons, r.gallops, r.merges);
            fflush(stdout);
        }
    }
    FIXED_MINRUN = 0;

    for (const string& kind : {string("average"), string("partial")}) {
        vector<int> src = generate(kind, 100000);
        for (int g : {7, 1000000000}) {
            GALLOP = g;
            Result r = measure(src);
            printf("gallop,%s,%d,%d,%d,%.1f,%lld,%lld,%d\n", kind.c_str(), 100000,
calc_minrun(100000), g,
                   r.time_ns, r.comparisons, r.gallops, r.merges);
```

```
            fflush(stdout);
        }
    }
    GALLOP = 7;
    return 0;
}
```

Файл src/modules/plot_benchmark.py

```
import csv
import math
import os

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CSV_PATH = os.path.join(BASE_DIR, "results.csv")
OUT_DIR = os.path.join(BASE_DIR, "plots")

BLUE = "#2563eb"
RED = "#dc2626"
GREEN = "#16a34a"
ORANGE = "#ea580c"

CASE_NAMES = {
    "best": "лучший случай",
    "average": "средний случай",
    "worst": "худший случай",
    "partial": "частично упорядоченный массив",
}


def load_data(path):
    rows = []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            row["n"] = int(row["n"])
            row["min_run"] = int(row["min_run"])
            row["gallop"] = int(row["gallop"])
            row["time_ns"] = float(row["time_ns"])
            row["comparisons"] = int(row["comparisons"])
            rows.append(row)
    return rows


def fit_scale(theory_vals, measured):
    theory_vals = np.asarray(theory_vals, dtype=float)
    measured = np.asarray(measured, dtype=float)
    denom = float(np.dot(theory_vals, theory_vals))
    if denom == 0:
        return 0.0
    return float(np.dot(theory_vals, measured)) / denom


def style_axes(ax, title):
    ax.set_title(title, fontsize=13, fontweight="bold", pad=12)
    ax.grid(True, which="both", alpha=0.25, linestyle="-")
    ax.legend(frameon=True, fontsize=10, loc="best")
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)


def save(fig, name):
    fig.tight_layout()
    out = os.path.join(OUT_DIR, name)
    fig.savefig(out, dpi=130)
    plt.close(fig)
```

```
    print(f"Сохранён график: {out}")


def plot_case(rows, case, label, model, filename):
    data = sorted((r for r in rows if r["experiment"] == "size" and r["case"] ==
case), key=lambda r: r["n"])
    n = np.array([r["n"] for r in data], dtype=float)
    t = np.array([r["time_ns"] for r in data]) / 1000.0
    theory = model(n)
    c = fit_scale(theory, t)
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(n, t, "o-", color=BLUE, markersize=6, linewidth=2, label="Эксперимент
(замер)", zorder=3)
    ax.plot(n, c * theory, "--", color=RED, linewidth=2, label=f"Теория {label}",
zorder=2)
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Размер массива n (элементов)")
    ax.set_ylabel("Время сортировки, мкс")
    style_axes(ax, f"TimSort, {CASE_NAMES[case]} — сравнение с теорией {label}")
    save(fig, filename)


def plot_comparisons(rows):
    fig, ax = plt.subplots(figsize=(8, 5))
    models = {
        "best": (GREEN, lambda n, mr: (2 * n - 1) / n, "2n - 1"),
        "average": (BLUE, lambda n, mr: mr / 4 + np.log2(np.maximum(n / mr, 1)),
"n·min_run/4 + n·log₂(n/min_run)"),
        "worst": (RED, lambda n, mr: mr / 2 + np.log2(np.maximum(n / mr, 1)),
"n·min_run/2 + n·log₂(n/min_run)"),
    }
    for case, (color, model, label) in models.items():
        data = sorted((r for r in rows if r["experiment"] == "size" and r["case"]
== case), key=lambda r: r["n"])
        n = np.array([r["n"] for r in data], dtype=float)
        mr = np.array([r["min_run"] for r in data], dtype=float)
        c = np.array([r["comparisons"] for r in data], dtype=float)
        ax.plot(n, c / n, "o-", color=color, markersize=5, linewidth=2, la-
bel=f"{CASE_NAMES[case].capitalize()} (замер)")
        ax.plot(n, model(n, mr), "--", color=color, linewidth=1.5, alpha=0.8, la-
bel=f"Теория {label}")
    ax.set_xscale("log")
    ax.set_xlabel("Размер массива n (элементов)")
    ax.set_ylabel("Сравнений на один элемент")
    style_axes(ax, "TimSort — число сравнений на элемент")
    ax.legend(frameon=True, fontsize=8.5, loc="upper left")
    save(fig, "comparisons.png")


def plot_minrun(rows, n_value, filename):
    data = sorted((r for r in rows if r["experiment"] == "minrun" and r["n"] ==
n_value), key=lambda r: r["min_run"])
    mr = np.array([r["min_run"] for r in data], dtype=float)
    t = np.array([r["time_ns"] for r in data]) / 1000.0
    best = data[int(np.argmin(t))]
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(mr, t, "o-", color=BLUE, markersize=6, linewidth=2, label="Эксперимент
(замер)", zorder=3)
    standard = next(r for r in rows if r["experiment"] == "size" and r["case"] ==
"average" and r["n"] == n_value)["min_run"]
    ax.axvline(standard, color=RED, linestyle="--", linewidth=2, la-
bel=f"Стандартный min_run = {standard}", zorder=2)
    ax.plot([best["min_run"]], [best["time_ns"] / 1000.0], "o", color=ORANGE, mark-
ersize=11, zorder=4,
            label=f"Минимум: min_run = {best['min_run']}")
    ax.set_xscale("log", base=2)
    ax.set_xlabel("min_run (логарифмическая шкала)")
    ax.set_ylabel("Время сортировки, мкс")
    style_axes(ax, f"TimSort, n = {n_value} — зависимость времени от min_run")
    save(fig, filename)
```

```
def plot_minrun_comparisons(rows, n_value, filename):
    data = sorted((r for r in rows if r["experiment"] == "minrun" and r["n"] ==
n_value), key=lambda r: r["min_run"])
    mr = np.array([r["min_run"] for r in data], dtype=float)
    c = np.array([r["comparisons"] for r in data], dtype=float)
    model = n_value * mr / 4 + n_value * np.log2(np.maximum(n_value / mr, 1))
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(mr, c, "o-", color=BLUE, markersize=6, linewidth=2, label="Эксперимент
(замер)", zorder=3)
    ax.plot(mr, model, "--", color=RED, linewidth=2, label="Модель n·min_run/4 +
n·log₂(n/min_run)", zorder=2)
    ax.set_xscale("log", base=2)
    ax.set_yscale("log")
    ax.set_xlabel("min_run (логарифмическая шкала)")
    ax.set_ylabel("Число сравнений")
    style_axes(ax, f"TimSort, n = {n_value} — число сравнений от min_run")
    save(fig, filename)


def main():
    if not os.path.exists(CSV_PATH):
        raise SystemExit(f"Не найден {CSV_PATH}. Сначала запустите ./benchmark >
{CSV_PATH}")
    os.makedirs(OUT_DIR, exist_ok=True)
    rows = load_data(CSV_PATH)
    plot_case(rows, "best", "O(n)", lambda n: n, "best.png")
    plot_case(rows, "average", "O(n log n)", lambda n: n * np.log2(n), "aver-
age.png")
    plot_case(rows, "worst", "O(n log n)", lambda n: n * np.log2(n), "worst.png")
    plot_comparisons(rows)
    plot_minrun(rows, 100000, "minrun_100000.png")
    plot_minrun(rows, 1000, "minrun_1000.png")
    plot_minrun_comparisons(rows, 100000, "minrun_comparisons.png")
    print(f"\nГотово. Графики в каталоге {OUT_DIR}")


if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ Б

Результаты тестирования

Таблица 4 - Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	calc_minrun(20), calc_minrun(16), calc_minrun(15), calc_minrun(33), calc_minrun(1000)	10, 8, 15, 9, 16	Вычисление min_run
2.	find_run для массива 1 -2 3 -4 5 6 -7 -8 9 -10 11 -10 -9	Возвращает 11, начало массива: 11 -10 9 -8 -7 6 5 -4 3 -2 1	Разворот строго возрастающего рана
3.	find_run для массива 5 -5 5 4 7	Возвращает 4, массив не меняется	Невозрастающий ран с равными модулями не разворачивается
4.	insertion_sort для массива 1 -3 2 3 -1 -2	-3 3 2 -2 1 -1	Устойчивость сортировки вставками
5.	gallop для массива 9 8 7 -7 5 3 с ключами 7, 10 и 1	4, 0, 6	Экспоненциальный и бинарный поиск длины блока
6.	20 1 -2 3 -4 5 6 -7 -8 9 -10 11 -10 -9 8 7 -7 -6 6 5 4	Part 0: 11 -10 9 -8 -7 6 5 -4 3 -2 1 Part 1: -10 -9 8 7 -7 -6 6 5 4 Gallops 0: 0 Merge 0: 11 -10 -10 9 -9 -8 8 -7 7 -7 6 -6 6 5 5 -4 4 3 -2 1 Answer: 11 -10 -10 9 -9 -8 8 -7 7 -7 6 -6 6 5 5 -4 4 3 -2 1	Пример 1 из условия: разворот рана, слияние без галопа
7.	16 -1 2 3 4 5 -6 7 8 -8 -8 7 -7 7 6 -5 4	Part 0: 8 7 -6 5 4 3 2 -1 Part 1: -8 -8 7 -7 7 6 -5 4 Gallops 0: 1 Merge 0: 8 -8 -8 7 7 -7 7 6 -6 5 -5 4 4 3 2 -1 Answer: 8 -8 -8 7 7 -7 7 6 -6 5 -5 4 4 3 2 -1	Пример 2 из условия: один вход в галоп
8.	16 3 -3 2 2 -2 1 1 -1 100 -10 -5 -4 3 3 2 1	Gallops 0: 3 Answer: 100 -10 -5 -4 3 3 3 -3 2 2 -2 2 1 1 -1 1	Вход в галоп засчитывается, даже когда половина закончилась
9.	16 10 -10 10 -10 -10 -10 10 10 11 10 -10 10 -10 -10 10 -10	Gallops 0: 1 Answer: 11 10 -10 10 -10 -10 -10 10 10 10 -10 10 -10 -10 10 -10	Галоп на элементах с равными модулями

№ п/п	Входные данные	Выходные данные	Комментарии
10.	32 10 -10 10 -10 -10 -10 10 10 10 11 10 -10 10 -10 -10 10 -10 25 -25 -15 13 -12 -11 11 -11 15 -13 12 11 -11 11 10	Part 0 … Part 3 выводятся до слияний, Gallops 0: 1, Gallops 1: 1, Gallops 2: 3, Merge 0: 25 -25 -15 13 -12 -11 11 -11 11 10 -10 10 -10 -10 10 -10	Четыре блока: порядок вывода и инвариант стека
11.	44 -9 -8 7 5 5 -4 -4 3 -3 2 -1 -10 9 -7 6 -4 -4 -3 2 -2 1 0 -8 -8 -8 -8 8 -7 5 -3 -1 -1 0 -9 9 -8 6 -6 -3 3 3 -1 1 -1	Gallops 0: 1, Gallops 1: 2, Gallops 2: 7	Много входов в галоп в последнем слиянии
12.	1 -43	Part 0: -43 Answer: -43	Массив из одного элемента
13.	3 10 -10 10	Part 0: 10 -10 10 Answer: 10 -10 10	Все модули равны: порядок не меняется
14.	10 -6 -6 -6 6 5 -5 5 4 -4 4	Part 0: -6 -6 -6 6 5 -5 5 4 -4 4 Answer: -6 -6 -6 6 5 -5 5 4 -4 4	Уже отсортированный массив - один ран
15.	200 случайных массивов длины до 2000, значения от -20 до 20; порог 16, галоп после 3 побед	Модули не возрастают, набор элементов совпадает с исходным	Корректность на больших данных